# Speed and Python surface across Pystachy's history

From 8 October to 11 November 2026, the share of CPython's own unit tests that Pystachy runs without hand edits rose from 0.3% to 6.7%, with no loss of speed relative to CPython; bounded `match` raised it to 8.3% on main. The added surface cost compile time and two benchmark regressions.

| | First day 8 Oct | v0.1.0-alpha.2 20 Oct | v0.4.0-alpha.5 11 Nov | main 8434a911 12 Nov |
|---|---|---|---|---|
| CPython test methods passing (of 5,905) | 15 | 371 | 397 | 492 |
| Language and builtins methods (of 2,445) | 14 | 211 | 218 | 310 |
| Adapted pyperformance ports working (of 25) | 0 | 20 | 25 | 25 |
| Unmodified pyperformance files compiling (of 52) | 0 | 7 | 7 | 7 |
| Microbenchmark speed index (× CPython) | 18.1 | 17.6 | 18.0 | 18.1* |
| Core ports speed index (× CPython) | — | 11.4 | 11.9 | 11.8* |
| AOT build of nbody (s) | 0.31 | 0.68 | 1.13 | 1.13* |
| Compiler bootstrap (s) | 12 | 164 | 246 | 239 |

* main was timed in a separate run together with alpha.5: its speed indices are alpha.5's scaled by the ratio measured in that run, and its build time is that run's measurement (pages 3 and 6).

Passing methods rose by about 26 a day for two weeks, then by 26 in total over three weeks while 31,000 lines were added. Language and builtins methods stayed at 218 from 28 October until bounded `match` added 92.

Speed held. The six day-one microbenchmarks ran 18.1 times faster than CPython on the first day and 18.0 times at alpha.5; the 16 pyperformance ports measured since 15 October went from 7.4 to 11.9 times. Two benchmarks regressed and are still slower on main: `float` by 37% since #1268 and `words` by 22% since #667.

Compile time absorbed the cost. The bootstrap grew from 12 s to 239 s in step with the compiler's size. An AOT build of the 82-line nbody benchmark grew from 0.31 s to 1.13 s, because every build re-optimizes the whole runtime; compiling nbody's own code still takes about 10 ms.

On main, 9% of the 5,389 rejected methods need runtime reflection, 42% a missing module or name, and 32% a class-model, inference or `match` feature a static compiler can implement. Removing one first error made only 0–43% of the targeted methods pass, because tests combine many dynamic features.


2 Recommendations
3 What was measured
4 Speed against surface
5 Surface over time
6 Speed over time
7 Speed by benchmark
8 Runtime regressions
9 Compile cost
10 Remaining blockers
11 Removing one blocker
12 Porting or compiler work
13 Data per commit

DECISION

# Recommendations and issues filed

Fix the two regressions, measure speed and surface at every release, and spend compiler work where it compounds across CPython's tests and ported code.

1. Recover the `float` and `words` slowdowns (#1300, #1301). Both mechanisms are identified: listguard's loop copies stop LLVM from inlining a constructor, and string methods count code points they do not need.
2. Time all 31 benchmark workloads in the weekly performance run (#1302), and record the CPython method triage and build cost for every release (#1303). Together they would have shown the regressions and the three-week plateau when they happened.
3. Infer more field and container types (#118, #618). They are the first error of 784 rejected methods, and type annotations or explicit types make up 161 of the 367 adaptations in the ported benchmarks.
4. Then extend the class model: classes defined inside functions (#1304); builtin, generic and multiple bases (#7, #962); and the remaining `match` patterns (#1305). Order this work by how many methods each change makes pass, measured with probes rather than first-error counts (#1306).
5. Before further class-model work, test one design change: in modules that opt in to dynamic `Any`, store fields whose type cannot be inferred as dynamic values instead of rejecting the module. Measure it on the CPython methods and on the 45 pyperformance files that do not compile unmodified.
6. Reduce the fixed AOT build cost (#706) and the super-linear growth of the GC-stressed self-compilation (#1181).
7. Leave `eval`, `exec`, `pickle` and frame introspection to embedded CPython (#170) and document them as deviations, rather than compiling them.

### Issues filed on 12 November 2026

| Issue | Kind | Title | Page |
|---|---|---|---|
| #1300 | Regression | Recover float's 37% AOT slowdown from listguard loop copies | 8 |
| #1301 | Regression | Recover words' 22% AOT slowdown from code-point strings | 8 |
| #1302 | Process | Time all pyperformance ports in the weekly performance run | 8 |
| #1303 | Process | Record CPython method triage and build cost per release | 5 |
| #1304 | Gap | Compile classes defined inside functions | 10–11 |
| #1305 | Gap | Support mapping, starred and class patterns in match | 11 |
| #1306 | Index | Rank Python surface work by measured CPython method unlocks | 10–11 |

Already tracked and not filed again: AOT build cost (#706), compile-stage timing (#1011), self-compilation cost (#1181), field inference (#118, #618), the class model (#7), standard-library and C-module gaps (#11, #491), `enum` (#610), `collections` (#256) and `array` (#716).

METHOD

# What was measured

Each sampled commit was built from source and given the same fixed inputs: CPython's own unit tests, pyperformance's benchmark files and a fixed set of timed programs.

### Commits

The history covers 34 commits on main from 8 October to 11 November 2026 (dates are UTC): the last commit of most days and 8 of the 15 release tags. Each commit was built in its own worktree by its own `make` bootstrap (CPython 3.13.18 hosting stage 1, LLVM 18.1.3). Main at `8434a911` (12 November, the alpha.6 feature freeze) was measured the same way in a follow-up.

### Workloads

| Workload | Inputs | Counts as passing |
|---|---|---|
| **CPython test methods** | 9,855 unittest methods extracted by `tools/cpython_test_methods.py` from 144 pinned CPython 3.13.16 test files; the 5,905 that complete under CPython 3.13.18 are the denominator, 2,445 of them in 60 language and builtins files | `pystachy run` (JIT) prints CPython's stdout and stderr and exits with the same status; a retry with promoting integers is allowed |
| **Unmodified pyperformance** | The 52 `run_benchmark.py` files of pyperformance 1.14.0, with pyperf's runner replaced | `pystachy ir` compiles the file |
| **Adapted benchmarks** | The 25 ports in `bench/pyperformance` and the six day-one `bench/*.py` microbenchmarks, unchanged across commits | AOT output equals CPython's; then timed |

The speed index is the geometric mean of per-benchmark speedups, where a speedup is CPython's time divided by Pystachy's AOT time on the same source. It uses a fixed set: the six microbenchmarks, or the 16 ports that worked at every sample from 15 October; `unpack_sequence` is excluded because AOT removes its timed loop.

### Timing

Timing ran after all builds on an otherwise idle 4-vCPU x86-64 VM, each executable pinned to one CPU. Every repetition ran CPython 3.13.18 and all commits in an order rotated by one position, and every output was checked. Values are medians of 5 runs (ports) or 7 (microbenchmarks): pyperf's timed region for ports, process wall time for microbenchmarks.

### Classifying rejections

A rejected method is classified by its first compiler error: library (a missing module or name), syntax (an unimplemented construct), typing (inference needs an annotation), dynamic (Python's dynamic class model and reflection) or other. Page 10 regroups the same errors by what removing them needs.

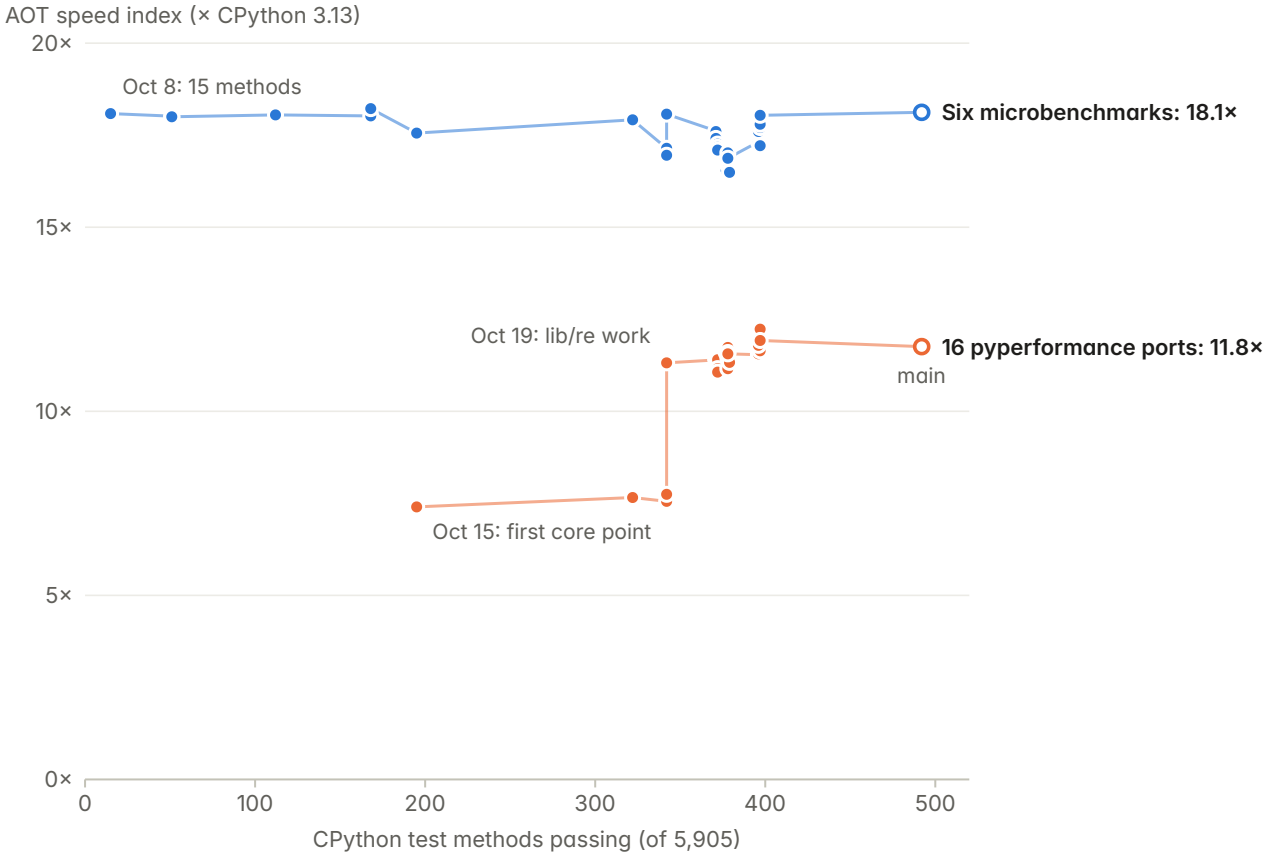
### Limits

Only a method's first error is visible, so first-error counts overstate what one fix unlocks (page 11). Methods ran through the JIT; AOT output was checked for the benchmarks only. Re-timing alpha.5 in a later run moved its core index by 5% and single ports by up to 40%, so cross-run comparisons use pairs timed together.

CENTRAL RESULT

# Speed against Python surface

Passing CPython test methods grew from 15 to 492 while the speed index held or rose; at the level of the index, the added surface did not cost run-time speed.




One point per sampled commit, joined in date order. Blue: the six day-one microbenchmarks; orange: the 16 pyperformance ports that worked at every sample from 15 October. The hollow points are main, derived from the paired run (page 6).

The microbenchmark index stayed between 16.5× and 18.2× at every sample, while passing methods went from 15 to 397. Individual microbenchmarks moved in both directions: `words` lost 24% and `fib` 8%, while `nbody` gained 18% and `sieve` 15%.

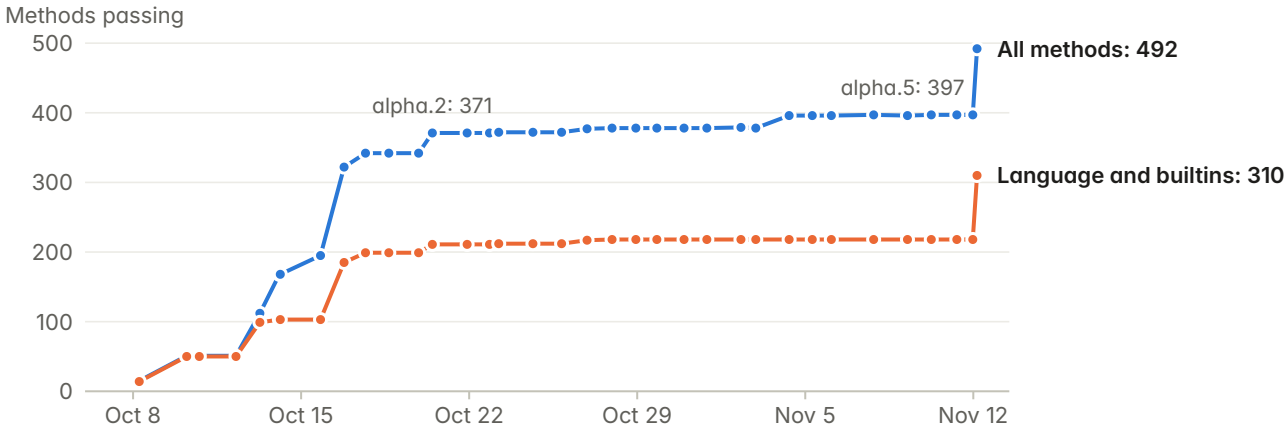
The core index rose from 7.4× to 11.3× between 15 and 19 October, during the largest surface gains. Most of that rise came from `lib/re` work that made the three regex ports 4.4 to 7.7 times faster (page 7). After 20 October both measures moved little until bounded `match` reached main.

The index is a geometric mean, so it hides regressions in single benchmarks (page 8), and it measures run time only; compile time is on page 9.

SURFACE

# Python surface over time

Passing methods grew by about 26 a day for two weeks, then by 26 in total over three weeks, until 95 more passed on main, 92 of them through bounded `match`.




CPython test methods that compile and match CPython's output, per sampled commit; 5,905 methods in total, 2,445 of them in language and builtins test files.

| Release | Date (UTC) | Methods passing | Language and builtins | Library | Ports working | Unmodified files |
|---|---|---|---|---|---|---|
| v0.1.0-alpha.1 | 14 Oct | 168 | 103 | 65 | 7 | 1 |
| v0.1.0-alpha.2 | 20 Oct | 371 | 211 | 160 | 20 | 7 |
| v0.1.0-alpha.3 | 23 Oct | 372 | 212 | 160 | 22 | 7 |
| v0.1.0-alpha.4 | 2 Nov | 378 | 218 | 160 | 23 | 7 |
| v0.2.0 | 6 Nov | 396 | 218 | 178 | 25 | 7 |
| v0.3.0 | 7 Nov | 397 | 218 | 179 | 25 | 7 |
| v0.4.0-alpha.2 | 10 Nov | 397 | 218 | 179 | 25 | 7 |
| v0.4.0-alpha.5 | 11 Nov | 397 | 218 | 179 | 25 | 7 |
| main `8434a911` | 12 Nov | 492 | 310 | 182 | 25 | 7 |

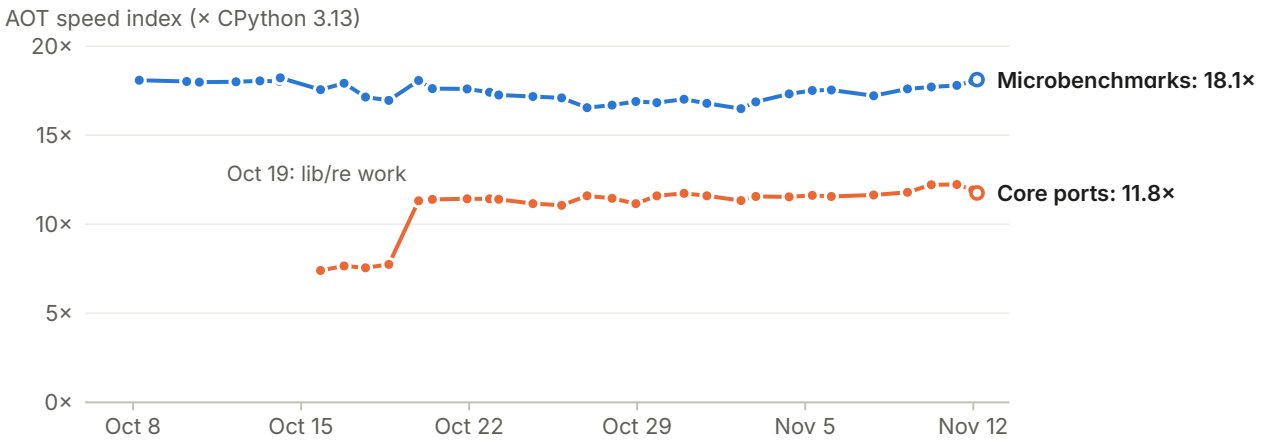
The plateau coincided with a shift of effort. After 20 October the compiler's share of changed lines fell from 16% to 5% and the library's from 8% to 3%, while benchmarks and examples rose from 3% to 31% and tools and CI from 4% to 12%. Over weeks three to five the compiler, library and runtime still grew by 31,000 lines, but only 26 methods began to pass: 0.1 to 1.4 per thousand lines, against 3.4 to 3.8 in the first two weeks.

The kind of blocker removed changed too. In the first two weeks, newly passing methods had been blocked by a missing library piece (169), syntax (96), typing (39) or the dynamic class model (19). From alpha.2 to alpha.5, none blocked by the dynamic model and 7 blocked by typing began to pass.

SPEED

# Speed over time

The core index rose from 7.4× to 11.3× CPython between 15 and 19 October and stayed between 11.1× and 12.2× afterwards; the microbenchmark index stayed near 18×.




Geometric mean of AOT speedups over CPython 3.13 per sampled commit. Hollow points: main, derived from the paired run.

| Sample | Date (UTC) | Commit | Core index | Microbenchmark index | Ports working |
|---|---|---|---|---|---|
| First day | 8 Oct | `4f7d51c5` | — | 18.09 | 0 |
| First core sample | 15 Oct | `bdc20a24` | 7.40 | 17.56 | 17 |
| After lib/re work | 19 Oct | `b040f45e` | 11.32 | 18.07 | 20 |
| v0.1.0-alpha.2 | 20 Oct | `afb59cd3` | 11.39 | 17.62 | 20 |
| v0.1.0-alpha.4 | 2 Nov | `9c8bc746` | 11.56 | 16.87 | 23 |
| Highest core index | 11 Nov | `d02995cb` | 12.23 | 17.80 | 25 |
| v0.4.0-alpha.5 | 11 Nov | `b2182a4f` | 11.92 | 18.04 | 25 |
| main (derived*) | 12 Nov | `8434a911` | 11.76 | 18.12 | 25 |

* alpha.5's index scaled by the main-to-alpha.5 ratio measured in one paired run: 0.986 for the core ports and 1.004 for the microbenchmarks. In that run the indices were 11.14 and 18.60 for main, against 11.29 and 18.52 for alpha.5.

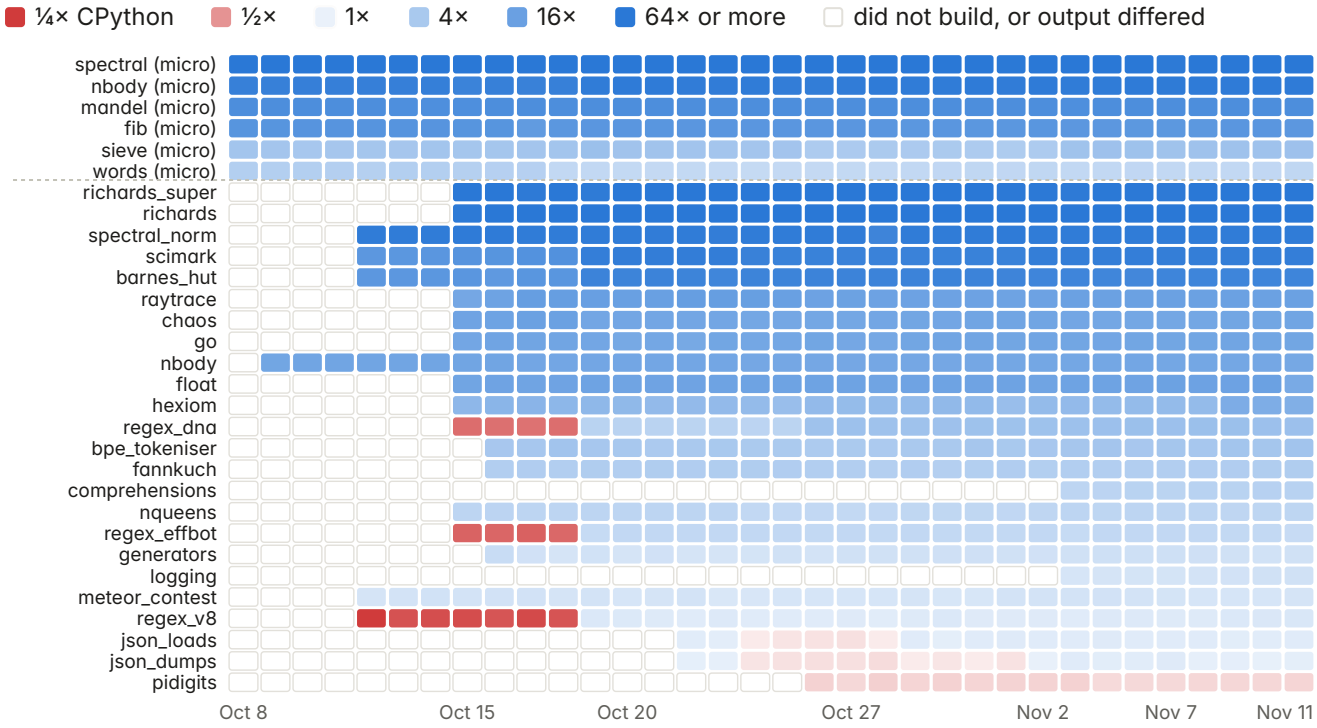
The fall from 12.2× to 11.9× on 11 November is mostly `float`: its speedup fell by a quarter at #1268, which moves a 16-port geometric mean by about 2%. Successive samples also move by up to about 3% in both directions; differences of that size cannot be told apart from noise in this setup.

On main, the paired run puts both indices within 2% of alpha.5, smaller than the 5% by which the same alpha.5 build differed between two runs. Main is therefore indistinguishable from alpha.5 in speed.

SPEED

# Speed by benchmark

Most ports kept the speed they reached in mid-October. The largest changes were the `lib/re` work on 19 October and two regressions that are still present on main.




AOT speedup over CPython 3.13 per benchmark (rows: microbenchmarks, then ports by speedup at alpha.5) and sampled commit (columns, 8 October to 11 November). Each row's source is fixed, so a change along a row is a compiler or library change.

| Benchmark | Interval | Time (ms) | Cause | Evidence |
|---|---|---|---|---|
| `regex_dna` | 18–19 Oct | 1,506 → 196 | `lib/re` matcher work (#676, #680, #682, #683, #685) | 32 merges; not bisected |
| `regex_effbot` | 18–19 Oct | 1,032 → 168 | Same `lib/re` work | Not bisected |
| `regex_v8` | 18–19 Oct | 1,047 → 235 | Same `lib/re` work | Not bisected |
| `scimark` | 18–19 Oct | 19.2 → 10.1 | Likely field TBAA (#724) or tuple returns by value (#727) | Not bisected |
| `barnes_hut` | 18–19 Oct | 24.9 → 13.4 | Likely the same two changes | Not bisected |
| `words (micro)` | 17–18 Oct | 59.3 → 77.7 | Code-point strings, stage 1 (#667) | Bisected to `a355c83f` |
| `float` | 11 Nov | 32.7 → 43.5 | listguard loop versioning (#1268) | Bisected to `b40ee875` |

At alpha.5, speedups range from 0.84× (`pidigits`) to 77× (`richards_super`). Seven ports run less than twice as fast as CPython: `pidigits`, `json_dumps`, `json_loads`, `regex_v8`, `meteor_contest`, `logging`, `generators`. The fastest are dominated by typed numeric loops and method calls; the slowest spend their time in library and runtime code for JSON, big integers, regular expressions, sets, generators and logging.

SPEED

# Two runtime regressions

`float` is 37% slower on main than before #1268, and `words` 22% slower than before #667. Both causes are identified (#1300, #1301), and neither benchmark is in the per-merge performance check.

| Benchmark | Before | At the merge | v0.4.0-alpha.5 | main | CPython 3.13 |
|---|---|---|---|---|---|
| `float` (pyperformance port) | 32.92 ms<br>0ec827ac | 43.62 ms<br>#1268, b40ee875 | 44.02 ms | 45.11 ms<br>+37% | 508.7 ms |
| `words` (microbenchmark) | 58.23 ms<br>6d4ed3d1 | 73.30 ms<br>#667, a355c83f | 71.76 ms | 71.19 ms<br>+22% | 179.2 ms |

AOT executables, medians of 15 interleaved runs pinned to one CPU on 12 November. "Before" is the merge's first parent. `float` reports pyperf's timed region; `words` reports process wall time.

### `float` and #1268

The slowdown is not extra work: instructions rose by 1.4% (350.3M to 355.3M), and page faults and peak memory are unchanged. #1268's listguard pass copies the loop `for i in range(n): points[i] = Point(i)` into a checked and an unchecked version, so the constructor `Point.__init__` gains a second call site and LLVM stops inlining it. With the compiler from before #1268, adding one never-executed second `Point(0)` call to the source produces the same out-of-line constructor and the same slowdown, from 34.1 ms to 44.8 ms. Any small callee in a versioned loop is exposed in the same way.

### `words` and #667

Instructions rose from 433M to 617M at #667 and are 539M on main. Two costs remain. First, `w.upper().startswith("L")` runs 400,000 times, and each call counts the code points of a freshly created string, although a call without explicit bounds does not need the count (64M instructions). Second, the heap peak rose from 34.9 MiB to 59.1 MiB, and one garbage collection taking 0.8 ms became two taking 5.0 ms; the cause of the growth is not yet known.

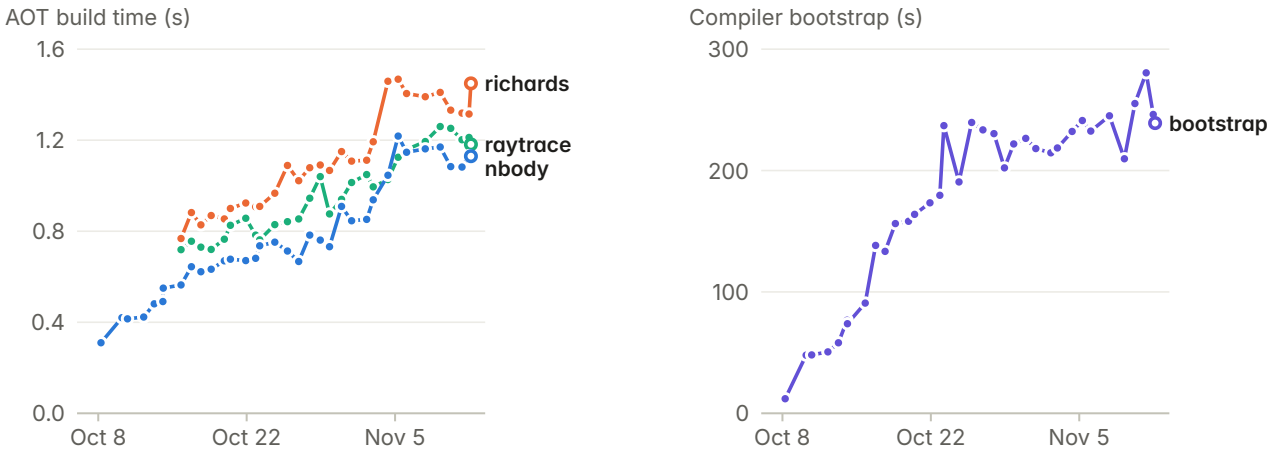
### Why they went unnoticed

The Performance smoke workflow times eleven workloads, and neither benchmark is among them; its weekly run compares main against a fixed reference on the same eleven. Instruction counts alone would also have missed `float`, whose time rose by 32% at #1268 for 1.4% more instructions. #1302 proposes weekly timing of all 31 workloads.

COST

# Compile cost

Compile time grew with the compiler and its runtime, not with the programs being compiled.




Left: `pystachy build` of three fixed programs, pinned to one CPU, medians of two runs. Right: the full `make` bootstrap of each commit. Hollow points: main, from the follow-up run.

| Commit | Frontend only (`pystachy ir`) | Full AOT build | Program IR (lines) |
|---|---|---|---|
| First day, `4f7d51c5` | 5–10 ms | 0.34 s | 946 |
| v0.1.0-alpha.2, `afb59cd3` | 9–15 ms | 0.69–0.80 s | 979 |
| main, `8434a911` | 9–12 ms | 1.19–1.37 s | 842 |

nbody (82 lines), three runs per commit pinned to one CPU on 12 November.

The bootstrap compiles the compiler with itself until its output stops changing, so its time follows the compiler's size: 12 s for 4,423 lines on the first day and 239 s for 66,836 lines on main, while the compiler's own IR grew from 52,488 to 912,733 lines. That is 2.7 to 4.8 s per thousand compiler lines at every sample. The GC-stressed self-compilation that CI runs grows faster than the compiler, about with its square (#1181).

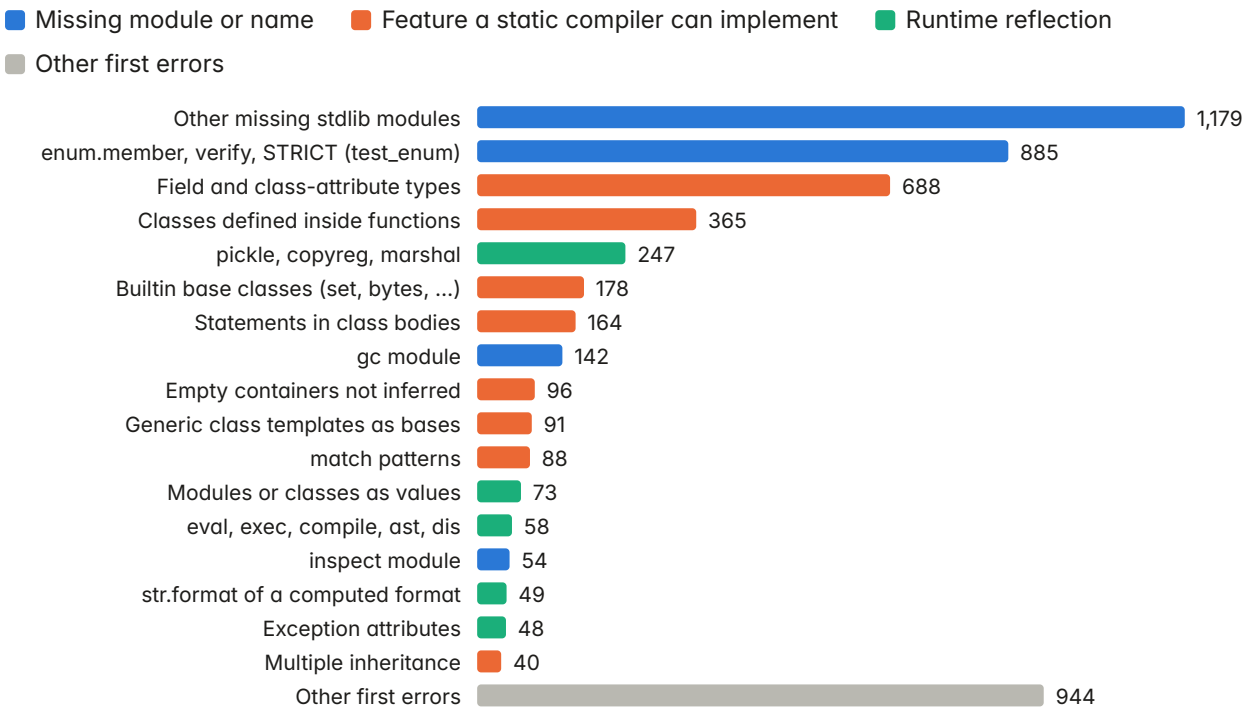
Compiling nbody's own code takes 5 to 15 ms at every measured commit, and its IR even shrank (table). The full AOT build grew from 0.31 s on the first day to 1.13 s at alpha.5 because each build links and re-optimizes the whole runtime, which grew from 2,154 to 14,800 lines. #706 measured 0.12 s for an nbody build with a precompiled runtime, but such executables ran 1.23 to 2.71 times slower. The JIT caches its compiled runtime, so running the small programs end to end stayed near 0.1 s (0.12 s on the first day, 0.10 s on main).

The bootstrap growth is the expected price of a self-hosting compiler fifteen times larger. The added AOT build time buys nothing for small programs, whose speed did not change, and is the most fixable of these costs.

SURFACE

# What blocks the remaining tests

On main, 5,389 of the 5,905 methods are rejected at compile time. 42% need a missing module or name, 32% a class-model, inference or `match` feature that a static compiler can implement, and 9% runtime reflection.




First compiler error of each rejected method on main `8434a911`, grouped by what removing it needs (`followup/scripts/groups.py`). Other missing stdlib modules include `configparser`, `decimal`, `_pyio`, `locale`, `pprint`, `queue`, `calendar` and `array`.

The largest single groups are not reflective. Field and class-attribute types that inference cannot determine stop 688 methods. Classes defined inside functions stop 365; in 78% of these methods the local classes capture no variable of the enclosing function. Builtin base classes, mostly `set` and `frozenset` in `test_set`, stop 178. Three names missing from Pystachy's `enum` (`member`, `verify` and `STRICT`) stop all 885 methods of `test_enum` at their imports.

Reflection is concentrated in five places: `pickle` round trips (247 methods in 28 files), modules or classes used as values (73), `eval`, `exec`, `compile` and the `ast` and `dis` modules (58), `str.format` with a computed format string (49), and exception attributes such as `__traceback__` (48).

The first-error classifier described on page 3 puts the class-model features together with reflection under "dynamic". By that classifier, dynamic and typing errors make up 81% of language-test rejections on main (76% at alpha.5), but most of them are features a static compiler can implement, not reflection.

SURFACE

# What removing one blocker unlocks

Removing a single first error rarely makes a test pass: probes made between 0% and 43% of the targeted methods pass, because the next error is usually another missing feature.

| Probe on main | Methods targeted | Then pass | Most common next error |
|---|---|---|---|
| Bounded `match` (#1272), methods rejected for `match` at alpha.5 | 216 | 92 (43%) | Starred, mapping and class patterns (71) |
| Local classes that capture nothing, moved to module scope | 218 | 23 (11%) | Builtin base classes (33), async functions (11) |
| A `gc` stand-in | 141 | 11 (8%) | Classes defined inside functions (51) |
| `inspect.cleandoc` | 52 | 0 | `compile()` of a source string (44) |
| CPython's pure-Python stdlib on the import path (alpha.5) | 2,429 | 0 | A missing module or name (2,031) |
| Every method opted in to dynamic `Any` | 5,905 | +7, −7 | No net change |

The `match` row compares alpha.5 with main. Every other probe removes one first error mechanically and reruns the methods through the JIT, on main or, for the standard-library probe, on alpha.5. Hoisting skipped methods whose local classes capture a variable or clash in name, and dropped those whose CPython output changed once hoisted.

Tests exercise features together. A test that defines a class inside a function often also subclasses a builtin type, uses async functions or returns values from unannotated functions, so those errors come next once the local class is moved out. `match` converted best because `test_patma` tests `match` itself.

Library work meets the same stacking. With CPython's own pure-Python standard library on the import path, none of the 2,429 library-blocked methods passed: they next needed C modules, such as `_contextvars` for `decimal` (260 methods) or `array` (84), names that Pystachy's own modules lack, or dynamic code inside CPython's library (219).

The dynamic `Any` opt-in does not reach these blockers. It gives dynamic storage to values a program declares as `Any`, while the blocked tests fail on inference and class features. Opting every method in made 7 more pass and rejected 7 that otherwise pass, through `math` functions applied to `Any` values, star-unpacking an `Any` into a call, and an assignment expression in a comprehension.

Ranking work by first-error counts therefore overstates its effect. #1306 lists the groups with their owners and asks for conversion probes before prioritizing.

ASSESSMENT

# Porting libraries versus extending the compiler

Adapting code works for a bounded set of libraries but does not raise the surface for unmodified code: all 25 adapted pyperformance ports run, against 7 of the 52 unmodified files.

| First error on main | Files | Examples |
|---|---|---|
| Missing module or name | 20 | `asyncio`, `multiprocessing`, `subprocess`, `decimal`, `array`, `hashlib`, `sqlite3`, `xml` |
| Typing | 11 | Field types (barnes_hut, bpe_tokeniser, float, go), a class attribute (chaos), empty containers (comprehensions, json_dumps) |
| Dynamic class model or reflection | 7 | `gc` (2), `pickle`, builtin or unsupported bases (deltablue, typing_runtime_protocols), generic templates as bases (richards, richards_super) |
| Other | 7 | Async functions (2), type annotations (2), a class-body default, `map()`, a function attribute |

The 45 unmodified pyperformance 1.14.0 files that `pystachy ir` rejects on main, by first error.

Type edits are the largest group of adaptations in the ported benchmarks: 91 of the 367 adaptation markers add annotations and 70 more make a type explicit, such as float literals or ranges turned into lists; replacing pyperf's runner accounts for 94. The `float` port, for example, annotates one parameter, three fields and one list. Better inference would let such files compile unmodified, and field inference is also the largest typing blocker in CPython's tests (688 methods).

Library breadth grows slowly through porting alone. From alpha.2 to main the shipped library grew by 8,400 lines (46,979 to 55,378) and 22 more library-test methods pass (160 to 182). Every port is also a copy to keep in step with its upstream.

For code that depends on reflection or C extensions, embedding CPython (#170) avoids both porting and compiler work: the compiled program calls CPython for those parts.

### Is the design too limiting?

Runtime reflection, which a typed, interpreter-free compiler cannot provide directly, accounts for about 9% of the rejected CPython tests. Passing most of CPython's own test suite would still need most of Python's dynamic class model, and each class-model feature makes few tests pass on its own, so that suite is a poor target. For the programs and libraries that users run, the combination that scales is better inference for unmodified code, ports for a bounded set of key libraries, and embedded CPython for the dynamic remainder. This is a judgment from the measurements above, not a measured result.

APPENDIX

# Measurements per commit

| Date | Commit | Release | Methods | Language | Ports | Unmod. | Core × | Micro × | nbody build (s) | Bootstrap (s) | Compiler lines |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 8 Oct | 4f7d51c5 | | 15 | 14 | 0 | 0 | — | 18.09 | 0.31 | 12 | 4,423 |
| 10 Oct | caf100d8 | | 51 | 50 | 2 | 0 | — | 18.02 | 0.42 | 48 | 16,730 |
| 10 Oct | 3542d631 | | 51 | 50 | 2 | 0 | — | 17.98 | 0.41 | 48 | 16,757 |
| 12 Oct | bd56b374 | | 51 | 50 | 2 | 0 | — | 18.00 | 0.42 | 51 | 16,923 |
| 13 Oct | 535a6f14 | | 112 | 99 | 7 | 1 | — | 18.05 | 0.48 | 58 | 20,558 |
| 14 Oct | 438565eb | | 168 | 103 | 7 | 1 | — | 18.02 | 0.49 | 77 | 24,619 |
| 14 Oct | aad4f1c2 | 0.1.0-alpha.1 | 168 | 103 | 7 | 1 | — | 18.22 | 0.55 | 74 | 24,648 |
| 15 Oct | bdc20a24 | | 195 | 103 | 17 | 1 | 7.40 | 17.56 | 0.56 | 91 | 27,383 |
| 16 Oct | 23791590 | | 322 | 185 | 20 | 6 | 7.66 | 17.92 | 0.64 | 138 | 38,465 |
| 17 Oct | 63977813 | | 342 | 199 | 20 | 6 | 7.55 | 17.14 | 0.62 | 133 | 40,339 |
| 18 Oct | bb5ac4ab | | 342 | 199 | 20 | 6 | 7.75 | 16.95 | 0.63 | 156 | 40,438 |
| 19 Oct | b040f45e | | 342 | 199 | 20 | 6 | 11.32 | 18.07 | 0.67 | 158 | 41,239 |
| 20 Oct | afb59cd3 | 0.1.0-alpha.2 | 371 | 211 | 20 | 7 | 11.39 | 17.62 | 0.68 | 164 | 45,928 |
| 21 Oct | 11722f0c | | 371 | 211 | 20 | 7 | 11.43 | 17.60 | 0.67 | 173 | 47,568 |
| 22 Oct | 7514bef7 | | 371 | 211 | 22 | 7 | 11.43 | 17.41 | 0.68 | 180 | 49,413 |
| 23 Oct | f5a09d42 | 0.1.0-alpha.3 | 372 | 212 | 22 | 7 | 11.40 | 17.26 | 0.74 | 237 | 49,848 |
| 24 Oct | 6f5b169b | | 372 | 212 | 22 | 7 | 11.16 | 17.17 | 0.75 | 191 | 51,021 |
| 25 Oct | de65c6fb | | 372 | 212 | 22 | 7 | 11.06 | 17.10 | 0.71 | 240 | 52,087 |
| 26 Oct | 368b12a5 | | 377 | 217 | 23 | 7 | 11.60 | 16.55 | 0.67 | 234 | 53,414 |
| 27 Oct | 48b28bb1 | | 378 | 218 | 23 | 7 | 11.45 | 16.69 | 0.78 | 230 | 54,608 |
| 28 Oct | 5e553e10 | | 378 | 218 | 23 | 7 | 11.16 | 16.89 | 0.76 | 202 | 55,383 |
| 29 Oct | d9de6b18 | | 378 | 218 | 23 | 7 | 11.60 | 16.83 | 0.73 | 222 | 56,642 |
| 30 Oct | 06e3ad5d | | 378 | 218 | 23 | 7 | 11.73 | 17.02 | 0.91 | 226 | 56,931 |
| 31 Oct | 07b26e02 | | 378 | 218 | 23 | 7 | 11.60 | 16.79 | 0.85 | 218 | 56,957 |
| 2 Nov | 14d4eb15 | | 379 | 218 | 23 | 7 | 11.33 | 16.49 | 0.85 | 215 | 57,717 |
| 2 Nov | 9c8bc746 | 0.1.0-alpha.4 | 378 | 218 | 23 | 7 | 11.56 | 16.87 | 0.94 | 219 | 57,902 |
| 4 Nov | a1436c85 | | 396 | 218 | 25 | 7 | 11.54 | 17.32 | 1.05 | 232 | 59,812 |
| 5 Nov | d8127936 | | 396 | 218 | 25 | 7 | 11.62 | 17.51 | 1.22 | 241 | 60,772 |
| 6 Nov | e7513fd5 | 0.2.0 | 396 | 218 | 25 | 7 | 11.56 | 17.54 | 1.15 | 232 | 60,772 |
| 7 Nov | c5b8b4fc | 0.3.0 | 397 | 218 | 25 | 7 | 11.64 | 17.21 | 1.16 | 245 | 64,218 |
| 9 Nov | d6b9b3f1 | | 396 | 218 | 25 | 7 | 11.79 | 17.60 | 1.17 | 210 | 64,534 |
| 10 Nov | 070c8137 | 0.4.0-alpha.2 | 397 | 218 | 25 | 7 | 12.22 | 17.71 | 1.08 | 255 | 65,199 |
| 11 Nov | d02995cb | | 397 | 218 | 25 | 7 | 12.23 | 17.80 | 1.08 | 280 | 65,632 |
| 11 Nov | b2182a4f | 0.4.0-alpha.5 | 397 | 218 | 25 | 7 | 11.92 | 18.04 | 1.13 | 246 | 66,345 |
| 12 Nov | 8434a911 | main | 492 | 310 | 25 | 7 | 11.76* | 18.12* | 1.13* | 239 | 66,836 |

* main, from the follow-up run (page 3). Dates are UTC commit dates. Methods and Language: CPython test methods passing, of 5,905 and 2,445. Ports: adapted pyperformance ports working, of 25. Unmod.: unmodified pyperformance files compiling, of 52.

## Data and reproduction

Inputs, raw results and scripts are on branch `claude/peaceful-faraday-2yr9q6` under `research/speed-vs-surface-2026-11-12`: charted rows in `data/`, timing, build, census and per-method results in `raw/`, the 12 November measurements on main in `followup/`, and the harness in `scripts/`. It is a research record, not for merging into main. Prepared with Claude Code assistance at the repository owner's request.